CS 305 Project One Template

Document Revision History

Version	Date	Author	Comments
1.0	5-27-25	Kyle Simonetti	

Client

Instructions

Submit this completed vulnerability assessment report. Replace the bracketed text with the relevant information. In this report, identify your security vulnerability findings and recommend the next steps to remedy the issues you have found.

- Respond to the five steps outlined below and include your findings.
- Respond using your own words. You may also include images or supporting materials. If you include them, make certain to insert them in the relevant locations in the document.
- Refer to the Project One Guidelines and Rubric for more detailed instructions about each section of the template.

Developer
Kyle Simonetti

1. Interpreting Client Needs
Determine your client's needs and potential threats and attacks associated with the company's application and software security requirements. Consider the following questions regarding how companies protect against external threats based on the scenario information:

- What is the value of secure communications to the company?
- Are there any international transactions that the company produces?
- Are there governmental restrictions on secure communications to consider?
- What external threats might be present now and in the immediate future?
- What modernization requirements must be considered, such as the role of open-source libraries and evolving web application technologies?

The value of secure communications in Artemis Financials case is very important, as they may be handling sensitive client information that could be a target for bad actors. Since the company handles individualized financials for their customers, this information could include Social Security Numbers, addresses, phone numbers, and other information that could potentially harm the customer if an attack was successful on the companies' data. The company does not state if it deals with international transactions but given that they develop financial plans and user a RESTful web API, it can be assumed that they would be open to working with clients from around the world. There may be government restrictions on secure communications to consider, such as the FTC's requirements to ensure customer information is protected from unauthorized access through secure communications. As this company works with sensitive customer information, they will have external threats present now and in the immediate future. These threats include injection attacks, DoS attacks, and phishing attacks. Modernization requirements that must be considered include dependency checks such as OWASP implementation to continue monitoring new and current vulnerabilities, keeping frameworks updated, and role-based access control to ensure only those who need to see sensitive information will see it. With all this said, data encryption, authentication systems, and input validation are all important tools to keep the data Artemis Financial has as secure as possible.

2. Areas of Security
Refer to the vulnerability assessment process flow diagram. Identify which areas of security apply to Artemis Financial's software application. Justify your reasoning for why each area is relevant to the software application.

The areas of security that apply to Artemis Financials' software application are input validation, APIs, cryptography, client / server, and code quality.
Input Validation – As user input will be required for customer information, improper input validation could lead to injection attacks from malicious inputs.
APIs – Each user should be validated, and permissions should be granted accurately. If permissions are mishandled, a user could gain access to another user's sensitive information. Users should also be validated to prevent another user from accessing someone else's account.
Cryptography – Because sensitive information is being sent over HTTPS and stored in databases, it is important for data at rest and data in transit to be encrypted to ensure the data remains safe even if it were to be stolen.

Client / Server – Data transmitted between the client and the server should be encrypted to prevent the sensitive data being transferred and ensures that privacy regulations are followed.
Code Quality – Making sure code quality remains high is important to prevent bugs that could lead to information being leaked or vulnerabilities being found. Coding standards should be followed, and code reviews should be done to catch issues early before live implementation.

3. Manual Review
Continue working through the vulnerability assessment process flow diagram. Identify all vulnerabilities in the code base by manually inspecting the code.

1. In the pom.xml file line 8, the Spring Boot version 2.2.4 is outdated, and should be updated to the latest version of 3.5
2. In the pom.xml file line 18, the Java 8 is outdated, and should be updated to the latest version of Java 24
3. In the pom.xml file line 30, the bouncycastle version 1.46 is outdated, and should be updated to the latest version of 1.8
4. In CRUDContoller.java line 13, there is no validation of the value variable
5. In Customer.java line 5, the account balance integer should be private
6. In myDateTime.java line 5-7, variables should be private
7. In mtDateTime.java, both methods used are not finished
8. In DocData.java line 27, the username and password should not be hardcoded
9. No authentication in GreeingController.java or CRUDController.java

4. Static Testing
Run a dependency check on Artemis Financial's software application to identify all security vulnerabilities in the code. Record the output from the dependency-check report. Include the following items:

- The names or vulnerability codes of the known vulnerabilities
- A brief description and recommended solutions provided by the dependency-check report
- Any attribution that documents how this vulnerability has been identified or documented previously

bcprov-jdk15on-1.46.jar	cpe:2.3:a:bouncycastle:bouncy_castle_for_java:1.46:*:*:*:*:*:*:* cpe:2.3:a:bouncycastle:legion-of-the-bouncy-castle:1.46:*:*:*:*:*:*:* cpe:2.3:a:bouncycastle:legion-of-the-bouncy-castle-java-crytography-api:1.46:*:*:*:*:*:*:*
CVE-2020-26939	In Legion of the Bouncy Castle BC before 1.61 and BC-FJA before 1.0.1.2, attackers can obtain sensitive information about a private exponent because of Observable Differences in Behavior to Error Inputs. This occurs in org.bouncycastle.crypto.encodings.OAEPEncoding. Sending invalid ciphertext that decrypts to a short payload in the OAEP Decoder could result in the throwing of an early exception, potentially leaking some information about the private exponent of the RSA private key performing the encryption.

	Bouncy Castle should be updated to the latest version https://nvd.nist.gov/vuln/detail/CVE-2020-26939
spring-boot-2.2.4.RELEASE.jar	cpe:2.3:a:vmware:spring_boot:2.2.4:release:*:*:*:*:*:* cpe:2.3:a:vmware:spring_framework:2.2.4:release:*:*:*:*:*:*
CVE-2023-20861 **CVE-2023-20883**	In Spring Framework versions 6.0.0 - 6.0.6, 5.3.0 - 5.3.25, 5.2.0.RELEASE - 5.2.22.RELEASE, and older unsupported versions, it is possible for a user to provide a specially crafted SpEL expression that may cause a denial-of-service (DoS) condition. In Spring Boot versions 3.0.0 - 3.0.6, 2.7.0 - 2.7.11, 2.6.0 - 2.6.14, 2.5.0 - 2.5.14 and older unsupported versions, there is potential for a denial-of-service (DoS) attack if Spring MVC is used together with a reverse proxy cache
	Spring Boot should be updated to the latest version https://nvd.nist.gov/vuln/detail/CVE-2023-6378 https://nvd.nist.gov/vuln/detail/CVE-2023-20861
logback-classic-1.2.3.jar **logback-core-1.2.3.jar**	cpe:2.3:a:qos:logback:1.2.3:*:*:*:*:*:*:*
CVE-2023-6378	A serialization vulnerability in logback receiver component part of logback version 1.4.11 allows an attacker to mount a Denial-Of-Service attack by sending poisoned data
	Logback should be updated to the latest version https://nvd.nist.gov/vuln/detail/CVE-2023-6378
log4j-api-2.12.1.jar	cpe:2.3:a:apache:log4j:2.12.1:*:*:*:*:*:*:*
CVE-2021-44832	Apache Log4j2 versions 2.0-beta7 through 2.17.0 (excluding security fix releases 2.3.2 and 2.12.4) are vulnerable to a remote code execution (RCE) attack when a configuration uses a JDBC Appender with a JNDI LDAP data source URI when an attacker has control of the target LDAP server. This issue is fixed by limiting JNDI data source names to the java protocol in Log4j2 versions 2.17.1, 2.12.4, and 2.3.2.
	Apache Log4j should be updated to the latest version https://nvd.nist.gov/vuln/detail/CVE-2021-44832
snakeyaml-1.25.jar	cpe:2.3:a:snakeyaml_project:snakeyaml:1.25:*:*:*:*:*:*:* cpe:2.3:a:yaml_project:yaml:1.25:*:*:*:*:*:*:*
CVE-2022-1471 **CVE-2022-3064**	SnakeYaml's Constructor() class does not restrict types which can be instantiated during deserialization. Deserializing yaml content provided by an attacker can lead to remote code execution. We recommend using SnakeYaml's

	SafeConsturctor when parsing untrusted content to restrict deserialization. We recommend upgrading to version 2.0 and beyond. Parsing malicious or large YAML documents can consume excessive amounts of CPU or memory.
	Update SnakeYaml to version 2.0 and beyond https://nvd.nist.gov/vuln/detail/CVE-2022-3064 https://nvd.nist.gov/vuln/detail/CVE-2022-1471
jackson-databind-2.10.2.jar	cpe:2.3:a:fasterxml:jackson-databind:2.10.2:*:*:*:*:*:*:*
CVE-2023-35116	jackson-databind through 2.15.2 allows attackers to cause a denial of service or other unspecified impact via a crafted object that uses cyclic dependencies. NOTE: the vendor's perspective is that this is not a valid vulnerability report, because the steps of constructing a cyclic data structure and trying to serialize it cannot be achieved by an external attacker.
	Update Jackson-databind to latest version https://nvd.nist.gov/vuln/detail/CVE-2023-35116
tomcat-embed-core-9.0.30.jar	cpe:2.3:a:apache:tomcat:9.0.30:*:*:*:*:*:*:*
CVE-2025-31651	Improper Neutralization of Escape, Meta, or Control Sequences vulnerability in Apache Tomcat. For a subset of unlikely rewrite rule configurations, it was possible for a specially crafted request to bypass some rewrite rules. If those rewrite rules effectively enforced security constraints, those constraints could be bypassed. This issue affects Apache Tomcat: from 11.0.0-M1 through 11.0.5, from 10.1.0-M1 through 10.1.39, from 9.0.0.M1 through 9.0.102. Users are recommended to upgrade to version [FIXED_VERSION], which fixes the issue.
	Update tomcat to the fixed version https://nvd.nist.gov/vuln/detail/CVE-2025-31651
hibernate-validator-6.0.18.Final.jar	cpe:2.3:a:redhat:hibernate_validator:6.0.18:*:*:*:*:*:*:*
CVE-2020-10693	A flaw was found in Hibernate Validator version 6.1.2.Final. A bug in the message interpolation processor enables invalid EL expressions to be evaluated as if they were valid. This flaw allows attackers to bypass input sanitation (escaping, stripping) controls that developers may have put in place when handling user-controlled data in error messages.
	Versions hibernate-validator 7.0.0.Alpha2, hibernate-validator 6.1.5.Final, hibernate-validator 6.0.20.Final all have a fix for this issue, so should be updated to one of these versions or later

	https://nvd.nist.gov/vuln/detail/CVE-2020-10693 https://bugzilla.redhat.com/show_bug.cgi?id=CVE-2020-10693
spring-web-5.2.3.RELEASE.jar **spring-webmvc-5.2.3.RELEASE.jar** **spring-context-5.2.3.RELEASE.jar** **spring-expression-5.2.3.RELEASE.jar** **spring-core-5.2.3.RELEASE.jar**	cpe:2.3:a:pivotal_software:spring_framework:5.2.3:release:*:*:*:*:*:* cpe:2.3:a:vmware:spring_framework:5.2.3:release:*:*:*:*:*:*
CVE-2020-5398 **CVE-2023-20863**	In Spring Framework, versions 5.2.x prior to 5.2.3, versions 5.1.x prior to 5.1.13, and versions 5.0.x prior to 5.0.16, an application is vulnerable to a reflected file download (RFD) attack when it sets a "Content-Disposition" header in the response where the filename attribute is derived from user supplied input. In spring framework versions prior to 5.2.24 release+ ,5.3.27+ and 6.0.8+ , it is possible for a user to provide a specially crafted SpEL expression that may cause a denial-of-service (DoS) condition.
	Update Spring Framework to latest version https://nvd.nist.gov/vuln/detail/cve-2020-5398 https://nvd.nist.gov/vuln/detail/CVE-2023-20863

5. Mitigation Plan

Interpret the results from the manual review and static testing report. Then identify the steps to mitigate the identified security vulnerabilities for Artemis Financial's software application.

Looking at the dependency check, all issues can be resolved if the tools being used are updated to their latest version, or in the hibernate validator's case, should be updated to one of the fixed versions. Looking at the manual review, Spring Boot, Java, and Bouncy Castle should be updated to the latest version. Input validation should be implemented in places such as CRUDController.java where user input is needed. To protect variables such as account-balance or myDateTime variables from being accessed anywhere, it should be set to private to prevent unauthorized changes to it. Incomplete methods should be commented on or completed to prevent attacks such as injections or errors by giving bad actors information of system details. In DocData.java, the username and password are hardcoded and should be prevented as it can be a significant risk if the source code becomes accessible somehow. Lastly, authentication and role based access control should be implemented to prevent unathorized users from seeing sensitive information that they should not have access to, as well as ensuring unauthorized users do not gain access to someone else's account which could have permissions to sensitive data.